

GLOBALRAIN

Informe de prácticas para un *software* seguro

Índice

INFORME DE PRÁCTICAS PARA UN *SOFTWARE* SEGURO 1

ÍNDICE 2

HISTORIAL DE REVISIÓN DE DOCUMENTOS 3

CLIENTE 3

INSTRUCCIONES 3

DESARROLLADOR 4

1. ALGORITMO DE CIFRADO 4

2. GENERACIÓN DE CERTIFICADOS 4

3. DESPLIEGUE DE CIFRADO 4

4. COMUNICACIÓN SEGURA 4

5. PRUEBAS SECUNDARIAS 4

6. PRUEBAS FUNCIONALES 4

7. RESUMEN 4

8. BUENAS PRÁCTICAS ESTÁNDAR DE LA INDUSTRIA 4

Historial de revisión de documentos

Versión	Fecha	Autor	Comentarios
1.0	[17/10/2026]	Ada Idalid Cortez Cota	Actualización de informe y elaboración de recomendaciones de seguridad profesionales.

Cliente



Desarrollador
Ada Idalid Cortez Cota

1. Algoritmo de cifrado

Para proteger la integridad de la información utilizada por la empresa, recomiendo utilizar SHA-256 (Secure Hash Algorithm 256-bit) como algoritmo hash. SHA-256 pertenece a la familia SHA-2, un conjunto de funciones hash criptográficas utilizadas para generar un valor que permita comprobar si determinada información ha sido modificada. Considero que este algoritmo es una opción adecuada para el proyecto debido a sus características de seguridad y a que continúa siendo ampliamente utilizado para la protección de la integridad de datos. Esta función Hash criptográfica fue diseñada por NSA (National Security Agency) y fue estandarizada por el National Institute of Standards and Technology (Encryption Consulting, s. f.).

Una función hash es un algoritmo que toma una entrada de cualquier tamaño y la transforma en una salida de longitud fija, conocida como digest o valor hash. En el caso de SHA-256, este resultado tiene una longitud de 256 bits, que normalmente se representa como una cadena de 64 caracteres hexadecimales. Además, SHA-256 funciona como una función unidireccional, lo que significa que no es viable obtener la información original a partir de su valor hash mediante un procedimiento práctico. Esto permite utilizar el resultado como una representación de los datos para posteriormente comprobar si estos permanecen íntegros.

Una característica especialmente importante al seleccionar un algoritmo hash es su resistencia a las colisiones. Una colisión ocurre cuando dos entradas diferentes producen exactamente el mismo valor hash. Esto representa un riesgo para la integridad de la información porque, si un atacante pudiera generar una entrada maliciosa que produjera el mismo valor hash que una entrada legítima, podría modificar la información sin que el mecanismo de comprobación detectara necesariamente la alteración. Por lo tanto, la resistencia a colisiones es una característica fundamental cuando el valor hash se utiliza para verificar que un archivo o dato no haya sido manipulado.

Por esta razón, no recomendaría utilizar algoritmos antiguos como MD5 o SHA-1, debido a que presentan vulnerabilidades conocidas relacionadas con la generación de colisiones. En el contexto de este proyecto, una vulnerabilidad de este tipo podría ser especialmente problemática si el cliente utiliza un valor hash para comprobar la integridad de una clave pública o de un archivo proporcionado por la empresa. Un atacante podría intentar reemplazar la información legítima por una versión maliciosa que produjera el mismo valor esperado, haciendo que el cliente confiara incorrectamente en la información modificada y comprometiendo la seguridad de las comunicaciones.

En conclusión, SHA-256 es mi recomendación para este proyecto porque ofrece una salida de 256 bits y una resistencia criptográfica adecuada frente a las vulnerabilidades que han hecho obsoletos a algoritmos como MD5 y SHA-1. Su resistencia a las colisiones y a otros ataques criptográficos permite utilizarlo como mecanismo para comprobar la integridad de los archivos y datos empleados por la empresa. Además, utilizar SHA-256 permite implementar una función hash moderna y ampliamente establecida, proporcionando un nivel de seguridad apropiado para este tipo de aplicación. Cuando el objetivo es mantenernos actualizados en prácticas de seguridad, es bueno utilizar entidades como la NSA como marcos de referencia para guiar nuestras recomendaciones y cumplir con estándares actuales.

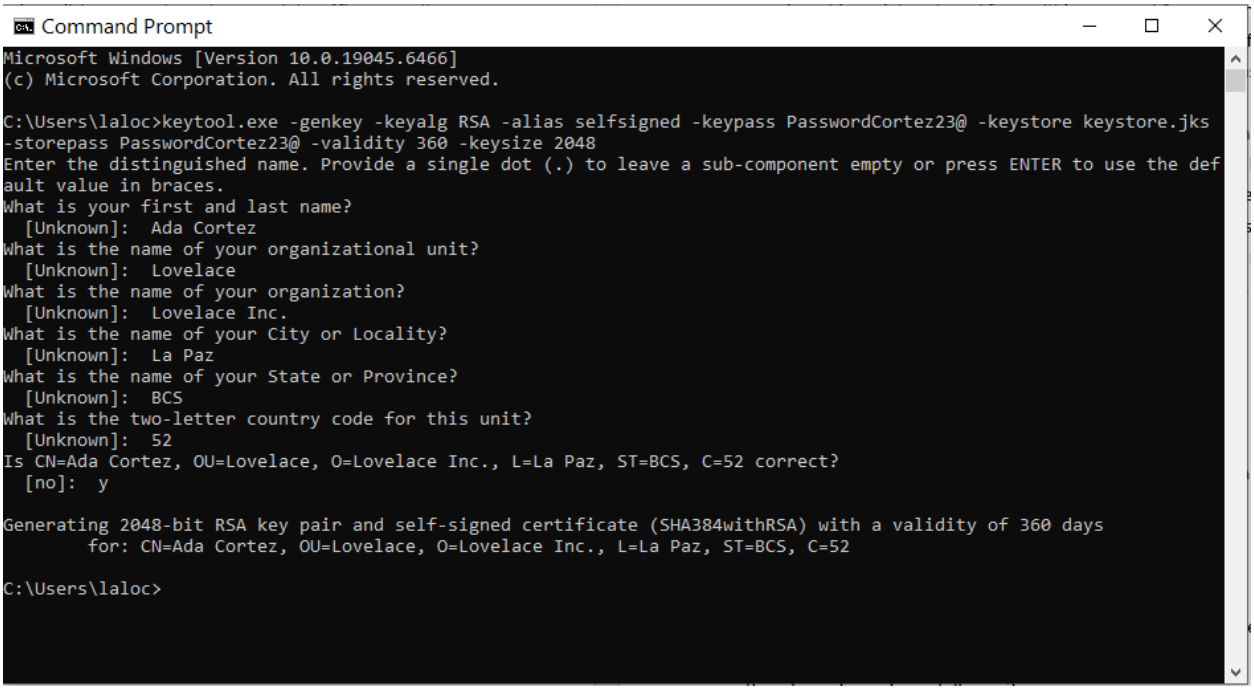
2. Generación de certificados

Una Autoridad de Certificación (AC) es "una empresa o entidad que ha sido autorizada por los navegadores para emitir certificados TLS/SSL" (DigiCert, s. f.). Es decir que está encargada de emitir y validar certificados digitales, documentos electrónicos que permiten asociar una identidad con una clave criptográfica utilizada para establecer comunicaciones seguras. Una de las principales razones para utilizar una AC es proporcionar confianza entre las partes que se comunican mediante una red, especialmente cuando existe una conexión entre un cliente y un servidor. Este tipo de certificados incrementa la confianza en un sitio web al permitir verificar que la identidad asociada con el dominio corresponde con la información certificada por una Autoridad de Certificación. Esto es importante para reducir el riesgo de phishing, una técnica de fraude en la que un sitio web malicioso intenta hacerse pasar por una entidad legítima, normalmente mediante una interfaz similar, con el objetivo de engañar al usuario para obtener información confidencial o realizar transacciones fraudulentas.

La AC verifica la identidad del propietario de un certificado y lo firma digitalmente, es decir, utiliza su propia clave privada para generar una firma que permite comprobar que el certificado fue emitido por esa autoridad y que no ha sido alterado. Esto resulta fundamental en protocolos como TLS (Transport Layer Security), un protocolo criptográfico utilizado para proteger la información transmitida entre sistemas mediante cifrado y autenticación. La AC no conoce la clave privada del servidor. El servidor demuestra que posee esa clave mediante una firma digital que puede ser verificada utilizando la clave pública. Posteriormente, la AC verifica que el solicitante también tiene control sobre la identidad que pretende certificar, como un dominio. Una vez realizadas estas verificaciones, la AC firma el certificado con su propia clave privada.

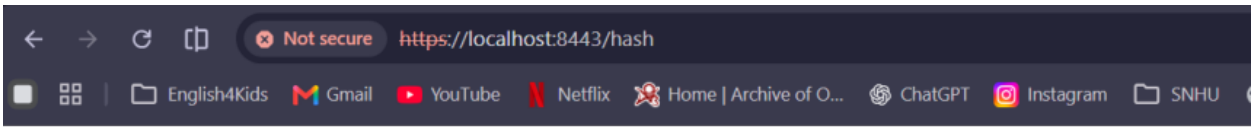
El uso de una AC también permite proporcionar autenticación, proceso mediante el cual se

verifica que una entidad es realmente quien afirma ser. Por ejemplo, cuando un navegador establece una conexión HTTPS con un sitio web, puede comprobar el certificado presentado por el servidor y determinar si fue emitido por una AC que reconoce como confiable. Esta relación de confianza se establece mediante una cadena de confianza, estructura en la que un certificado puede ser validado a través de una AC intermedia hasta llegar a una AC raíz, que es una autoridad cuyo certificado es confiado directamente por el sistema operativo o navegador. De esta manera, el usuario no necesita establecer manualmente una relación de confianza con cada servidor individual, ya que puede confiar en los certificados emitidos por autoridades reconocidas. Un caso potencial en el que las AC intermediarias son útiles es si "una autoridad de certificación principiante que recién inicia sus operaciones puede no contar aún con las cualificaciones necesarias para emitir un certificado raíz. Por ello, utilizará los servicios de otra CA bien establecida y vinculará sus certificados a un certificado raíz válido, formando así la cadena de confianza" (Keyfactor, s. f.).



3. Despliegue de cifrado

Para verificar la correcta implementación del algoritmo hash en la aplicación, se desarrolló un mecanismo de suma de comprobación utilizando SHA-256. La siguiente captura muestra la ejecución del sistema procesando una cadena de datos única que incluye mi nombre, generando de manera exitosa el resumen criptográfico de 64 caracteres hexadecimales. Esto garantiza que cualquier modificación no autorizada en los datos transferidos pueda ser detectada inmediatamente, cumpliendo con los estándares de integridad exigidos por el cliente.



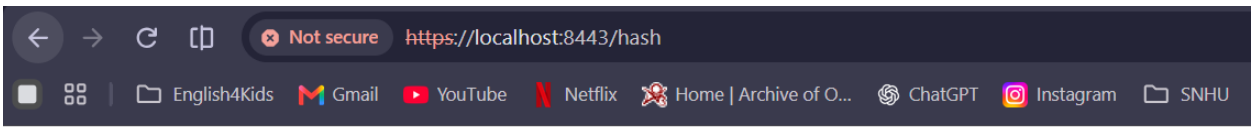
Cadena de datos: Ada Cortez

Algoritmo Hash: SHA-256

Suma de comprobación (Checksum): 3f2d8a5e88d78016de4c54ca9520344ef36d6e8c3ddfd670c18ecbae1e1ecb96

4. Comunicación segura

La siguiente captura de pantalla demuestra la conectividad exitosa al endpoint /hash mediante el protocolo HTTPS en el puerto 8443. Se puede observar la advertencia 'Not secure' emitida por el navegador; este comportamiento es completamente esperado al utilizar un certificado autofirmado en un entorno de desarrollo local, ya que el navegador no posee una cadena de confianza validada por una Autoridad de Certificación pública externa. No obstante, el tráfico se encuentra cifrado de extremo a extremo bajo SSL/TLS, cumpliendo con los requisitos de seguridad establecidos para el transporte de datos.

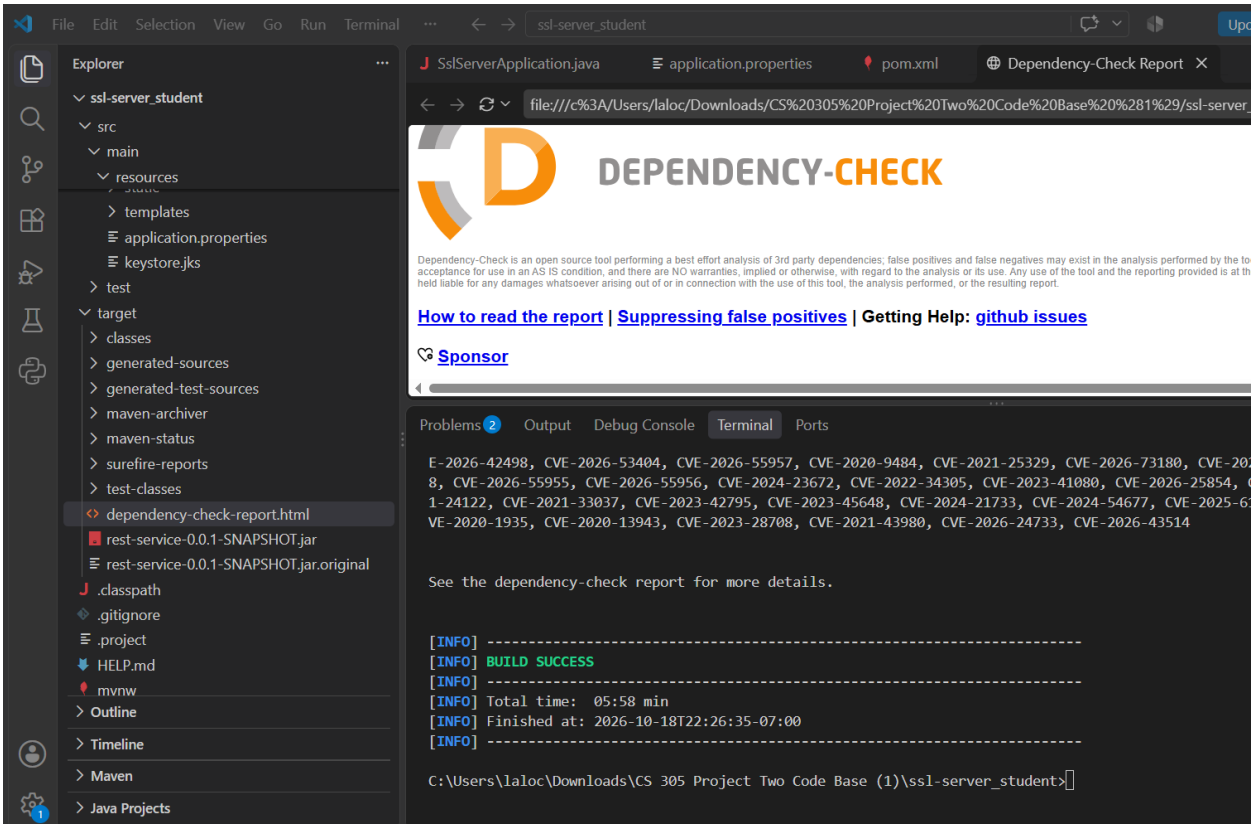


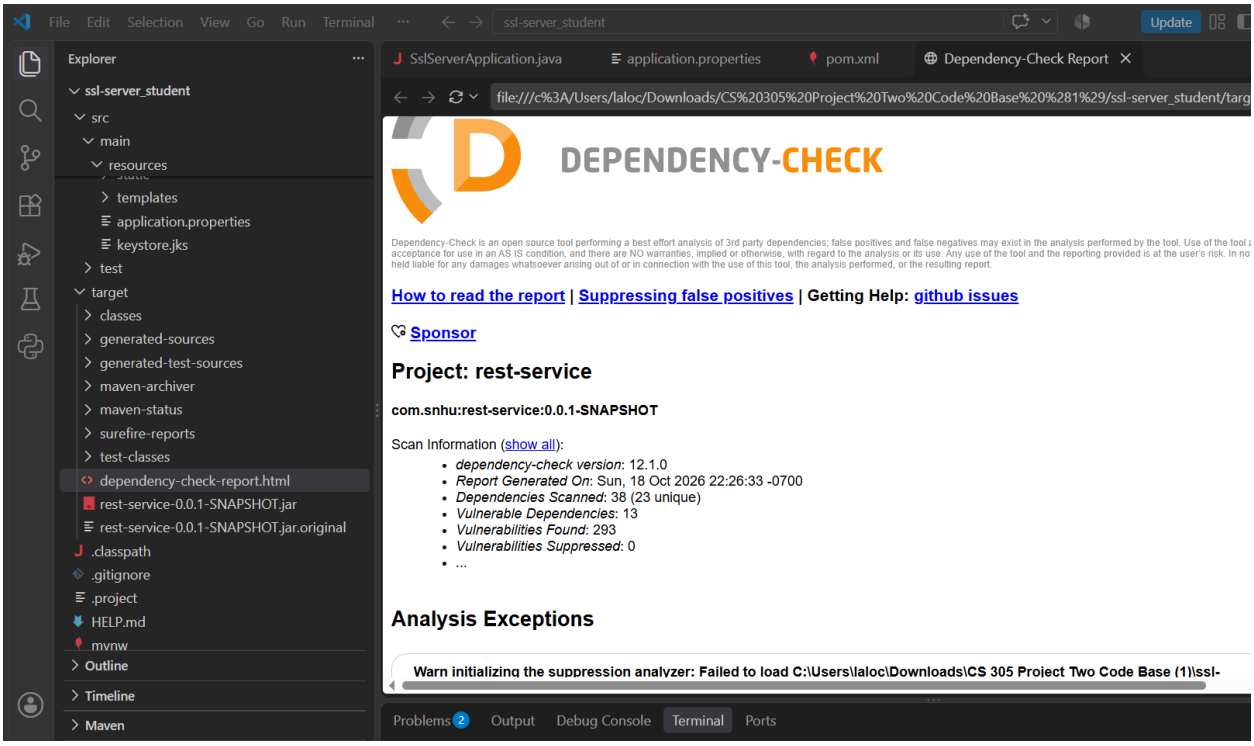
Cadena de datos: Ada Cortez
Algoritmo Hash: SHA-256
Suma de comprobación (Checksum): 03b363fee7d1b0b969281af2e56b09bad4e694b22e7077455b454ea8a695c62e

5. Pruebas secundarias

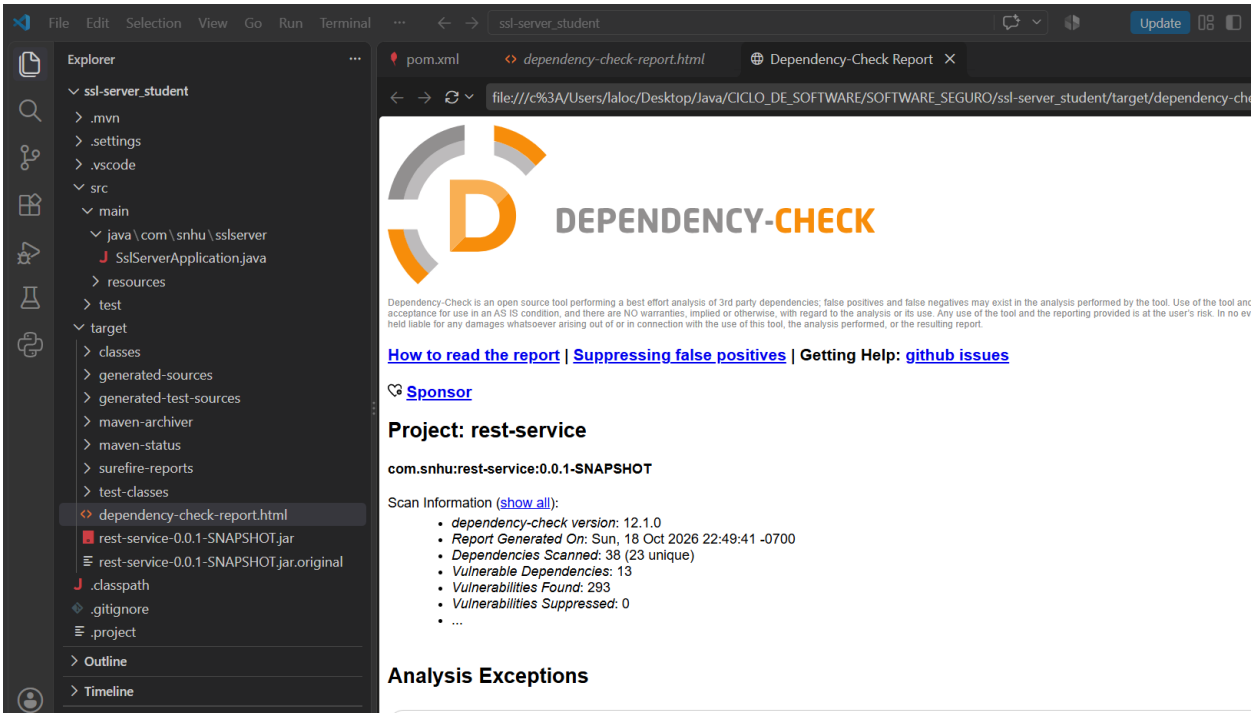
A continuación capturas de pantalla del código refactorizado ejecutado sin errores y del informe de Dependency-Check.

Se ejecutó la herramienta OWASP Dependency-Check integrada a través de Maven para realizar un análisis de vulnerabilidades sobre las librerías y componentes del proyecto. El reporte generado (mostrado en las evidencias) permite identificar el estado de seguridad de las dependencias, cumpliendo con los controles preventivos requeridos para mitigar riesgos e integrar prácticas de seguridad en el ciclo de desarrollo de Artemis Financial.





[Dependency-Check: Después]



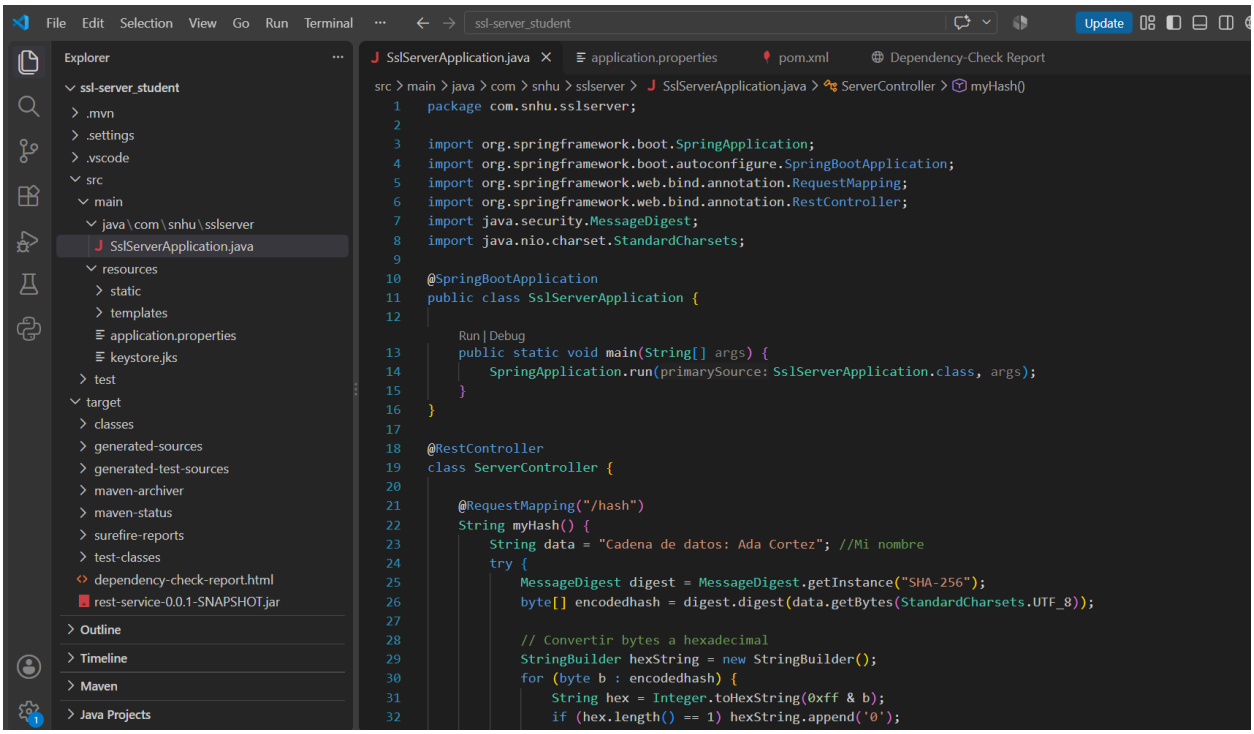
[Dependency-Check: Antes]

Adicionalmente, al comparar los resultados del dependency check en el proyecto antes y después de mis modificaciones en el código, podemos observar que no se introdujeron vulnerabilidades adicionales (en lo que concierne las dependencias).

6. Pruebas funcionales

Captura de pantalla del código refactorizado ejecutado sin errores.

Durante la revisión manual inicial del código base proporcionado por Artemis Financial, se identificó que el sistema carecía de un mecanismo robusto para garantizar la integridad de los datos mediante funciones hash criptográficas y no contaba con una implementación estructurada para el control seguro de operaciones hash. Como parte de la refactorización para un software seguro, se incorporó la clase MessageDigest utilizando el algoritmo estándar SHA-256, asegurando que los datos de entrada procesados generen una suma de comprobación única y verificable de manera segura. La siguiente evidencia muestra el código refactorizado funcionando sin errores de compilación ni ejecución.



Respecto a las vulnerabilidades comunes, se concluye que no aplica ningún riesgo de inyección (como SQL Injection o Inyección de Código), debido a que el sistema no recibe, procesa ni concatena ningún tipo de input o parámetro externo proveniente de peticiones de usuario; los datos están completamente controlados de forma interna. La única consideración de seguridad radica en que, al ser una cadena estática, cualquier cambio futuro en los datos requerirá una modificación directa en el código fuente en lugar de una recepción dinámica, lo cual mantiene el entorno seguro pero limitado a un escenario de prueba controlado.

De igual forma, no aplican vulnerabilidades de exposición accidental de información sensible (Information Disclosure), ya que la función hash resultante genera una representación criptográfica unidireccional de la cadena de texto definida internamente, sin revelar datos confidenciales, credenciales o contraseñas en el cuerpo de la respuesta HTTP devuelta al cliente.

Por otro lado, respecto a la robustez del algoritmo criptográfico, no aplica ninguna debilidad por uso de hashes obsoletos o inseguros (tales como MD5 o SHA-1), debido a que se especificó explícitamente el uso de SHA-256 mediante la clase MessageDigest, cumpliendo con los estándares actuales de seguridad contra ataques de colisión. Tampoco se identifican problemas de denegación de servicio (DoS) por consumo excesivo de recursos, dado que la operación hash se ejecuta sobre una cadena corta y fija, manteniendo un tiempo de respuesta sumamente bajo y predecible para el servidor web de la aplicación.

7. Resumen

Se refactorizó el código proveído para lograr crear una aplicación que establece una conexión segura mediante el protocolo HTTPS con la web, y se utilizó para hacer display de una prueba donde se imprime el resultado de una función hash, que según el contexto de Artemis Financial, será uno de los niveles de seguridad en nuestro sistema para cerciorarnos de que las comunicaciones se mantengan con integridad y podamos verificar que los datos no fueron manipulados en tránsito. Al consultar el diagrama de flujo del proceso de evaluación de vulnerabilidades, las áreas principales que se abordaron durante la refactorización fueron el control de la integridad criptográfica y la protección de datos en la capa de transporte, asegurando que la transmisión de información no quede vulnerable a interceptaciones o alteraciones.

Para añadir estas capas de seguridad a la aplicación, se siguió un proceso estructurado que comenzó con la configuración de certificados digitales para habilitar comunicaciones cifradas vía SSL/TLS en el servidor. Posteriormente, se integró en la lógica del controlador el uso del algoritmo SHA-256 a través de MessageDigest para generar sumas de comprobación (checksums) únicas sobre los datos, garantizando que el sistema cuente con los mecanismos necesarios para validar la autenticidad del contenido. Finalmente, se ejecutaron pruebas de análisis estático con OWASP Dependency-Check para verificar que las dependencias del proyecto no introdujeran vulnerabilidades conocidas en la arquitectura.

8. Buenas prácticas estándar de la industria

La aplicación de las buenas prácticas de la industria se enfocó en preservar y reforzar la seguridad existente del software mediante el uso de estándares criptográficos vigentes y alineados a las recomendaciones del NIST y la OWASP. Se mantuvo la estructura modular de Spring Boot, evitando prácticas inseguras como la concatenación de datos o el uso de algoritmos obsoletos (como MD5 o SHA-1), y se implementaron bloques adecuados de manejo de excepciones para prevenir caídas imprevistas o comportamientos indeterminados en el servidor.

Referencias

DigiCert. (s. f.). ¿Qué es una autoridad de certificación (CA)? https://www.digicert.com/es/faq/compliance/what-is-a-certificate-authority-ca

Encryption Consulting. (s. f.). SHA-256. https://www.encryptionconsulting.com/es/centroeducativo/sha-256/

Keyfactor. (s. f.). La diferencia entre los certificados raíz y los certificados intermedios. https://www.keyfactor.com/es/education-center/the-difference-in-root-certificates-vs-interme diate-certificates/